PROJECT EXPLAINER

ClaimDesk QA

What it is, what it proves, and how to run it. Written for someone who has never seen the code.

Repository https://github.com/codernik-dev/playwright-python-sdet-framework

Live test report https://codernik-dev.github.io/playwright-python-sdet-framework/

Author Nikesh Walia, QA / Test Automation Engineer

LinkedIn https://www.linkedin.com/in/nikesh-walia/

WHAT IS INSIDE

1 What this is

ClaimDesk QA is an automated testing project. It contains a small insurance claims web application, built only so there is something realistic to test, and a much larger body of software whose single job is to prove that application behaves correctly, over and over, without a human clicking anything. That second part is the actual piece of work. It exists for two reasons. The first is practical: to put every technique a modern quality engineer is expected to use (driving a real web browser, calling a web service directly, reading the database, running the whole thing inside containers, publishing a report automatically) into one place where it is running rather than described. The second is honesty. Every number quoted anywhere in this project came from a command that was actually run, and anything that was written but never executed is labelled as unverified, in public, where a reader can check it.

Repository	github.com/codernik-dev/playwright-python-sdet-framework
Live test report	codernik-dev.github.io/playwright-python-sdet-framework/
Written in	Python, using pytest, Playwright, httpx and PostgreSQL
Licence	MIT (free to read, copy and reuse)
Author	Nikesh Walia, QA and Test Automation Engineer moving toward SDET

2 The two halves, and which one is the point

The first half is ClaimDesk, a pretend insurance claims portal. It is not a demo screen with three buttons. It has real logins, three kinds of user (a customer, a handler, an administrator), a claim that moves through a fixed sequence of states, money limits that must not be exceeded, an audit trail that records who changed what, and a payout ledger. It carries 58 tests of its own. All of that exists so that the tests have something with genuine rules to break.

The second half is the test framework, and that is the deliverable. It is an installable Python package of 43 files and roughly 3,900 lines, plus a suite of tests that is larger still (32 files, roughly 5,500 lines). It is written the way production software is written, with strict type checking, documented decisions, and a rule that the tools enforce rather than a rule people are asked to remember.

The everyday version: ClaimDesk is the car on the test track, and the framework is the instrumented rig that drives it, records everything, and decides whether it passed. Nobody hires the car.

> **The one line worth remembering**
>
> The application is a fixture. The framework is the deliverable.

3 Three angles of attack, and two more besides

The same application is attacked from three independent directions, because each one can see something the other two are blind to. A suite that only tests one way will happily report green while the product is broken somewhere that way cannot reach.

- Through the browser, 32 tests. Playwright (a tool that drives a real browser engine under program control) clicks, types and waits exactly as a person would. This is the only layer that catches what only a person at a screen meets: a button that never becomes clickable, a form that lets you submit a date it should have refused, a page that shows one customer's claim to somebody else. The nightly run does this in Chromium, Firefox and WebKit, which are the engines behind Chrome, Firefox and Safari.
- Through the API, 157 tests. An API is the machine-to-machine doorway a web application exposes, the thing the screen itself talks to. Testing it directly, with no browser involved, catches the rules the screen is hiding, because a screen can grey out a button while the service behind it still cheerfully accepts the request. It is also fast, which is why the largest share of the suite lives here.
- Through the database, 28 tests. The database is where the data actually lands. This layer catches the gap between the screen saying it worked and the record having been written correctly: a missing audit row, a payout that does not reconcile, a status stored that the rules say is impossible. It reads and never writes.
- End to end, 4 tests. One journey that crosses all three in a single test: do it in the browser, confirm it through the API, verify it in the database. Deliberately few. These are the slowest and most brittle tests in any suite, so they are kept for the handful of journeys that genuinely need all three at once.
- The framework's own unit tests, 130. The test code is code, and untested test code is the most dangerous code in the building, because when it is wrong it does not crash, it reports green.

That last point invites an obvious objection, so it is worth stating plainly rather than leaving a reader to find it. Of the 351 tests, 130 test the framework itself and 221 are aimed at the application. Both numbers are published, in that order, in the project's measurement notes.

4 The rule that makes it professional

There is one rule, and most of the design follows from it: the tests may never import the application's own code. Importing means reaching inside the product and calling its internal functions directly. It is tempting, because it is quicker and it makes tests pass. It is also how a test suite ends up proving only that the code agrees with itself. If the application's own rule about, say, the maximum payout is wrong, a test that borrows that same rule will confirm the wrong answer forever. So this framework keeps its own separate copy of the business rules, and reaches the application only the way a real customer or a real integration would, over HTTP, through a browser, or with read-only database queries.

A rule that depends on people remembering it is not a rule. This one is enforced three ways, none of which rely on good intentions.

- The linter refuses it. A linter is a tool that reads the code and rejects patterns you have banned. Write an import of the application anywhere in the test suite and the check fails before the change can be committed, with a message that says what to do instead.
- The database account cannot write. The account the tests use holds permission to read and nothing else. An attempt to insert a row fails with an insufficient privilege error from PostgreSQL itself. The tests physically cannot manufacture data that the application would never have produced.
- The container does not contain the application. The image that runs the tests in the automated pipeline does not include the application's code at all, and the pipeline asserts that importing it fails inside that image. That assertion is checked against a case that should succeed, so the check itself cannot quietly pass for the wrong reason.

```
The message the linter prints if you try:

  The test framework must NEVER import the application under
  test. Use the HTTP API, the browser, or read-only SQL
  instead. See docs/adr/0002-black-box-boundary.md
```

5 The numbers

Every figure below was produced by running something, on the machine named at the bottom of the table. None of them is an estimate.

Tests passing	351
Split by layer	130 framework, 157 API, 32 browser, 28 database, 4 end-to-end
Aimed at the application	221 of the 351
Tests that expect a refusal	136 negative, 28 boundary, 23 authorisation, 22 integrity
Run time, one test at a time	24.88 seconds
Run time, four at a time	16.82 seconds (1.48 times faster)
Run time, sixteen at a time	23.81 seconds (slower than four)
Flakiness	0 failures across 10 consecutive runs (3,510 test executions)
Inside containers	351 passed in 36.39 seconds
The application's own tests	58
Type checking	mypy in strict mode across 94 files
Framework size	43 files, about 3,900 lines
Test suite size	32 files, about 5,500 lines
Written up in	14 phase documents covering 18 build phases, 9 decision records
Measured on	AMD Ryzen 7 5800H, 16 logical cores, Python 3.12, PostgreSQL 17.6

- 351 tests passing. The whole suite, every layer, with nothing skipped and nothing quarantined.
- 136 negative tests. A negative test checks that the application refuses something it ought to refuse. This is the ratio that matters most, because a suite that only proves the happy path also passes against an application that accepts absolutely everything.
- 24.88 seconds against 16.82 seconds. Running four tests at the same time rather than one after another makes the suite 1.48 times faster. Worth having, and modest, which is the honest result.
- 23.81 seconds on sixteen. This is the interesting number. The common advice is to use every processor core the machine has. On this suite sixteen workers are barely faster than running one test at a time, because three costs do not divide: starting each worker, preparing test data, and waiting on the database. Use all your cores is a guess, not a strategy, and it took ten minutes to measure.
- 0 failures across 10 consecutive runs. A flaky test is one that fails at random without the product having changed, and flakiness is the fastest way for a team to stop trusting its own tests. Ten runs of 351 tests is 3,510 individual executions with no unexplained failure.

- 351 passed in 36.39 seconds inside containers. Containers are self-contained packages that run identically on any machine, which is what turns it works on my laptop into a checkable claim rather than an excuse.
- mypy in strict mode across 94 files. Type checking catches whole categories of mistake before the code is ever run. Strict is the demanding setting, and it is applied to the framework, the tests and the application alike.

6 Three real bugs it found, in plain English

Every one of these was found by running the thing, not by reading it. Two of the three were faults in the tests rather than in the application, which is the more useful kind to be able to talk about, because a wrong test is invisible until somebody goes looking.

A check that passed for the wrong reason, and would have kept passing even if the login had been removed altogether. The suite asked the application for a list of claims while presenting no credentials at all, and expected to be turned away. It was not turned away, it got a normal answer, which looks exactly like a serious security hole. Repeating the same request by hand proved the application was fine. The cause was in the test tooling: the HTTP client the tests use quietly keeps cookies (the small token a website leaves behind so it remembers you are signed in), so a leftover cookie from an earlier test had signed the supposedly anonymous request in. Every identity now gets its own client that stores nothing, so an unauthenticated request really is unauthenticated.

A bug that could only fail for five and a half hours a day. ClaimDesk refuses to accept a claim for an accident dated in the future, and the suite checks both sides of that line, that today is accepted and tomorrow is refused. Both checks asked whichever machine they happened to be running on what today's date was, which was fine while there was only one machine. Then the application moved into a container set to UTC while the tests ran on a machine set to Indian Standard Time, five and a half hours ahead. Between midnight and half past five in the morning, in that one environment, the test machine's today was the application's tomorrow, so the accepted side would fail while its matching partner kept passing and hid half the problem. Both sides now state their time zone explicitly instead of asking the machine.

A large slowdown that was not the application's fault, and the shape of the numbers proved it. Every API test was spending around 0.4 seconds getting ready and 0.02 seconds actually checking anything, while the application itself answered in 2.8 milliseconds. The giveaway was that the overhead was identical everywhere: real performance problems in a product are lumpy, they cluster around one slow query or one slow page. Measurement found the cost inside the test code, where creating an HTTP client rebuilds the entire security handshake configuration from scratch (355 milliseconds each time) and the framework was building one per identity per test. Sharing a single configuration took a full run from 107.19 seconds to 30.14 seconds, and two permanent tests now hold it in place, one that the configuration is reused and one that it still verifies certificates, so that the tempting wrong fix (switching certificate checking off to reclaim the time) fails the suite instead of passing it.

7 How ready it is, stated honestly

This section is deliberately specific, because a readiness claim with no edges is not a readiness claim.

Working and proven, as of today:

- All three automated pipelines on GitHub are green: the gate that runs on every change, the nightly run across three browser engines, and the container run.
- The nightly run publishes its report automatically, and that report is live and browsable on the web. It is a real generated report from a real run, not a screenshot.
- The full suite has run green on Windows, on a Linux GitHub runner, inside Docker Compose (351 passed in 36.39 seconds), and on a real Jenkins controller.
- 0 unexplained failures across 10 consecutive runs.
- The build log records 114 individual items as verified, each against the command that verified it.
- Every item on the build log's list of open items carried forward is now closed.

Genuinely not proven, and worth naming:

- The automatic retry path has never actually fired. If a test fails in the automated pipeline, the suite is set up to retry it a limited number of times and report it as flaky rather than simply red. The logic that decides how many retries a test gets is unit tested in both directions, and the summary it prints has been reviewed, but the path itself has never run for real, because nothing has flaked in the pipeline. Reviewed is not the same as observed.
- On Jenkins, the reporting plugin's own publishing step was never exercised. When the pipeline ran, the fallback path ran instead. That fallback is a verified degradation path, which is worth something, but the primary path is untested. The same pipeline was only ever run on a Windows agent, so its Linux branch is unexercised too.

The build log still carries ten unverified markers in total. Eight of them now carry an explicit note recording what later superseded them, and the two above are the ones that remain genuinely open.

> **Why say any of this**
>
> A test suite is a claim about a product. A suite that cannot say which of its own claims have actually been executed is asking to be trusted rather than showing why it should be. Both items above could have been left out at no cost and nobody would have noticed. They are in because the opposite habit, quietly rounding written up to working, is exactly the habit that lets a green dashboard sit on top of a broken product.

8 How to run it

Three routes, in increasing order of effort. Pick one. Route zero takes about thirty seconds and installs nothing.

ROUTE ZERO: just look, install nothing.

- The published test report, at https://codernik-dev.github.io/playwright-python-sdet-framework/ . It shows 351 passed and 0 failed from the most recent nightly run, the test names in full readable sentences, the environment the run happened in (including passwords shown as masked rather than printed), and a link back to the exact automated run that produced it. It is a live JavaScript application rather than a static page, so give it a moment to load. It is the Chromium leg of the nightly run: publishing all three browser engines to one address would have them overwrite each other, so one is published deliberately.
- The repository itself, at https://github.com/codernik-dev/playwright-python-sdet-framework . The README carries the same numbers as section 5 above, each one next to the command that produced it.

ROUTE A: containers only. This is the easiest way to run everything yourself, and it works the same on Windows, macOS and Linux. The only thing you need installed is Docker (Docker Desktop on Windows or macOS, Docker Engine on Linux). No Python, no database, no browser downloads.

1. Get a copy of the repository.
2. Build the images. The word test in the build command is required rather than optional: without it the test image is skipped, and a later run will quietly use an older one.
3. Start the database and the application, and let them report healthy. Nothing here sleeps for a fixed number of seconds; each service is gated on a real health check.
4. Run the suite. Expect 351 passed. The measured time for this route was 36.39 seconds.
5. While it is up, open the application yourself in a browser and sign in (details further down).
6. Stop everything and remove the throwaway database when you are finished.

```
git clone https://github.com/codernik-dev/playwright-python-sdet-framework
cd playwright-python-sdet-framework

docker compose --profile test -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up -d db sut
docker compose -f docker/docker-compose.yml run --build --rm tests

# when you are done
docker compose -f docker/docker-compose.yml down -v
```

- The service is called sut (system under test), not app. That is a bug fix rather than a naming preference: .app is a real internet suffix that browsers are hard-coded to require encryption for, so while the service was named app every browser test failed with an encryption error against a plain server.
- Results are written to allure-results and artifacts on your own machine rather than left inside the container, so a failing run leaves its evidence behind.

ROUTE B: on your own machine, with Python and PostgreSQL. This is the route the project was developed on, and the instructions in the repository are written for Windows PowerShell. It needs Python 3.11 to 3.13 and the PostgreSQL server programs. It does not need PostgreSQL installed as a running service: the helper script builds its own disposable database on port 55432 and never touches any database you already have.

```
# Windows PowerShell, inside the repository folder
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[dev,app]"
playwright install chromium     # once; pip fetches no browsers

Copy-Item .env.example .env     # then set the two passwords

.\scripts\local_db.ps1 start    # disposable database, port 55432
python -m uvicorn claimdesk.main:app --app-dir app `
    --host 127.0.0.1 --port 8000
```

Then, in a second terminal window, with the same virtual environment activated:

```
pytest -m framework -q   # 130 framework tests, no app or database
pytest -m api -q         # 157 tests against the API
pytest -m ui -q          # 32 browser tests
pytest -m db -q          # 28 read-only database checks
pytest -m e2e -q         # 4 full journeys across all three layers

pytest -q -n 4           # everything, four at a time, the fast way
```

Operating system caveats, all of them found by actually trying it:

- On macOS and Linux, replace the first three lines with python3.12 -m venv .venv, then source .venv/bin/activate, then cp .env.example .env. The README carries this path too, along with the shell versions of the suite runner, the report builder and the quality gate.
- There is no shell version of the disposable-database script. It is PowerShell only, and it looks for the Windows PostgreSQL programs. On macOS and Linux, use Route A to get the database and the application running, or point the settings at a PostgreSQL you already have. The 130 framework tests need neither, so they run anywhere with nothing but a virtual environment.
- The bootstrap, benchmark and Docker helper scripts are PowerShell only. The quality gate, the suite runner and the report script also ship as shell scripts, where the options are environment variables rather than flags, for example WORKERS=4 ALLURE=1 ./scripts/run_suite.sh
- On Windows PowerShell 5.1, activating the virtual environment can be blocked by the default script execution policy. Allow it for that window only with Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
- If pip fails at the first step with a message about an invalid TLS certificate bundle path ending in ca-bundle.crt, some PostgreSQL installers set that path to a folder that does not exist. Clear it with Remove-Item Env:CURL_CA_BUNDLE -ErrorAction SilentlyContinue

- If PostgreSQL is not installed at all, you still need only the server programs. Unpack a copy and point the script at it by setting the PGBIN environment variable to the folder holding pg_ctl.exe before running the database script.

The demo application, once it is running by either route:

Sign in at	http://127.0.0.1:8000/login
Email	customer@example.com
Password	Passw0rd!seed
Interactive API documentation	http://127.0.0.1:8000/docs

Seeing the report for a run you did yourself. The test run writes raw results, and a separate step renders them into the browsable report. That renderer needs Java installed. If you would rather not install Java, there is a plain single-file HTML report instead.

```
# Windows
.\scripts\run_suite.ps1 -Workers 4 -Allure
.\scripts\report.ps1

# macOS and Linux
WORKERS=4 ALLURE=1 ./scripts/run_suite.sh
./scripts/report.sh

# no Java available
pip install -e ".[html]"
pytest --html=report.html --self-contained-html
```

9 Where to look in the repository

README.md	The short version of all of this, with each number next to the command that produced it
docs/progress.md	The build log, item by item, separating what was verified from what was not
docs/phase-15-measurement.md	Every number, how it was produced, and how easily numbers like these can lie
docs/debugging.md	The runbook: the pipeline is red at three in the morning, do these things in this order
docs/adr/	Nine decision records, one per significant choice, each with the reasoning and what it cost
docs/phase-1-design.md	The plan written before any code, including the risk register and a 69-case test matrix
docs/interview-preparation.md	The questions this project invites, including the five the author would struggle with
src/claimdesk_qa/	The framework itself: settings, HTTP client, database access, page objects, reporting
tests/	The tests, one folder per layer: framework, api, ui, db, e2e
app/	ClaimDesk, the practice application, a fixture rather than the deliverable
docker/	The container definitions and the compose file that brings the whole environment up at once
.github/workflows/	The three automated pipelines: the change gate, the nightly cross-browser run, the container run
Jenkinsfile	The Jenkins pipeline, executed on a real controller rather than written and assumed
scripts/	Setup, quality gate, disposable database, suite runner, report, benchmark

10 What this project is meant to show

Anyone can produce a green test run. What is harder, and what this project argues for, is the habit underneath it. Decide what a test is allowed to know, then make the tools enforce that instead of trusting people to remember it. Measure before optimising, and publish the measurement even when the honest answer is a modest 1.48 times rather than an impressive one. Treat a passing test as suspicious until you know why it

passes, because the anonymous request that came back with a normal answer and the date check that could only break for five and a half hours a day were both green the whole time. Keep what has been proven and what has merely been written in two separate columns, and leave the unproven items in the document where a reader will find them, which is the entire reason section 7 names two of them. A project that can only report good news is not reporting.

> **The two links, once more**
>
> Live report: https://codernik-dev.github.io/playwright-python-sdet-framework/ . Source: https://github.com/codernik-dev/playwright-python-sdet-framework . Questions and criticism are both welcome, and the second is more useful.